

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code
ECE 4124

Course Title
Digital Signal Processing Sessional

Lab Reports

Submitted to	Submitted by
Md. Faysal Ahamed Assistant Professor Dept of ECE, RUET & Moloy Kumar Ghosh Lecturer Dept of ECE, RUET	Md. Tajim An Noor Roll: 2010025

Table of Contents

Exp No.	Experiment Name	Date
1	Study of Impulse, Step, and Ramp signals and their inter-relationships	July 5, 2025
2	Study of Shifting, Folding & Scaling of Discrete Signals	July 21, 2025
3	Study of Linear and Circular Convolution, Cross-Correlation, and Auto-Correlation of Discrete Signals	August 2, 2025
4	Performing DFT to compute Convolution Between Two Signals	September 13, 2025
5	Implementation of FFT and IFFT on Discrete Signal	October 11, 2025
6	Design & Analysis of a Low-Pass FIR and IIR Filter & Their frequency Response Comparisons	October 18, 2025

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code
ECE 4124

Course Title
Digital Signal Processing Sessional

Experiment Date: July 5, 2025,
Submission Date: July 21, 2025

Lab Report 1:
Study of Impulse, Step, and Ramp signals and their inter-relationships

Submitted to
Md. Faysal Ahamed
Lecturer
Dept of ECE, Ruet

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Theory	1
Used Tools	2
Code & Output	3
Unit Impulse Signal	3
Unit Step Signal	3
Unit Ramp Signal	4
Conversion	5
Discussion	7
Conclusion	7
References	7

Study of Impulse, Step, and Ramp signals and their inter-relationships

Theory

In digital signal processing, several basic discrete-time signals serve as the building blocks for analyzing and synthesizing more complex signals [1]. The three most important among these are:

1. Unit Impulse Signal ($\delta[n]$):

The unit impulse signal, $\delta[n]$, is defined as:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

- Serves as the identity element in convolution operations [1].
- Has a value of 1 only at $n = 0$ and 0 elsewhere.
- Commonly used to determine a system's impulse response.

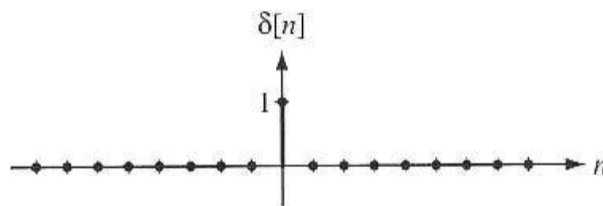


Figure 1: Unit Impulse Signal $\delta[n]$

2. Unit Step Signal ($u[n]$):

The unit step signal, $u[n]$, is defined as:

$$u[n] = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

- Switches from 0 to 1 at $n = 0$ and remains at 1 for all subsequent n .
- Can be interpreted as the running sum of the impulse signal: $u[n] = \sum_{k=-\infty}^n \delta[k]$ [1].

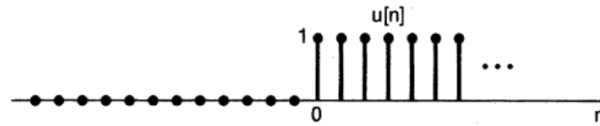


Figure 2: Unit Step Signal $u[n]$

3. Unit Ramp Signal ($r[n]$):

The unit ramp signal, $r[n]$, is defined as:

$$r[n] = \begin{cases} 0, & n < 0 \\ n, & n \geq 0 \end{cases}$$

- Increases linearly with n starting from zero.
- Represents the cumulative sum (discrete integration) of the unit step signal: $r[n] = \sum_{k=-\infty}^n u[k]$ [1].

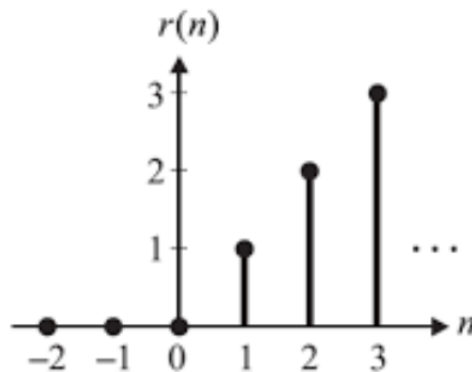


Figure 3: Ramp Signal $u[n]$

Used Tools

- Matlab
- LaTeX

Code & Output

Unit Impulse Signal

```
1 % Length of impulse response
2 N = 11; % Total number of samples
3 % Generate unit impulse using impz
4 x = impz(1, 1, N); % delta[n] from n = 0 to N-1
5 % Time index
6 n = 0:N-1;
7 % Plot the impulse
8 stem(n, x, 'filled', 'LineWidth', 2);
9 grid on;
10 xlabel('n (discrete time)');
11 ylabel('Amplitude');
12 title('Discrete Unit Impulse Signal (delta[n])');
13 ylim([-0.2 1.2]);
```

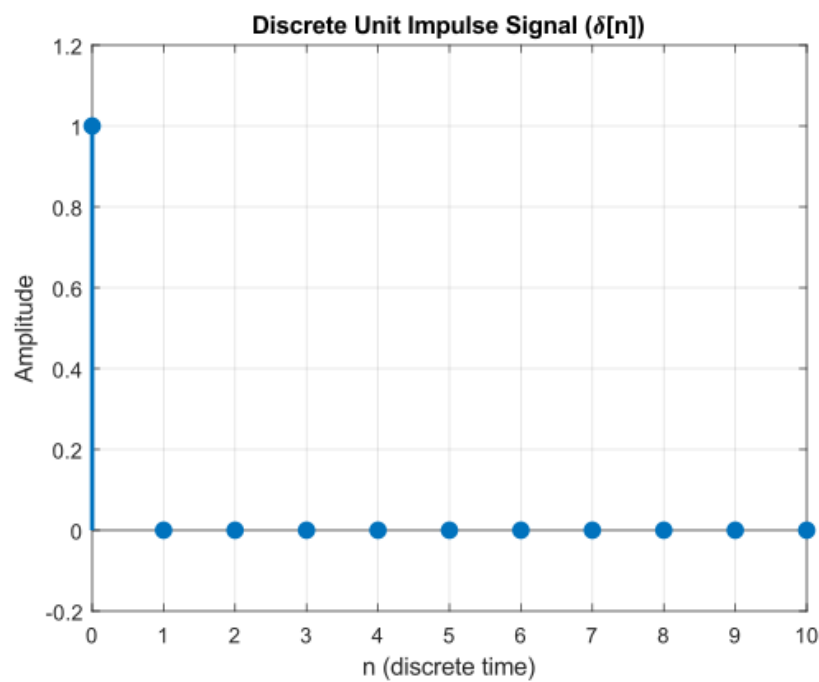


Figure 4: Unit Impulse Signal $u[n]$

Unit Step Signal

```
1 % Define discrete time index
2 n = -5:5; % From -5 to 5
3 % Generate discrete unit step signal
4 x = double(n >= 0); % u[n] = 1 for n >= 0, 0 otherwise
5 % Plot the step signal
```

```

6  stem(n, x, 'filled', 'LineWidth', 2);
7  grid on;
8  xlabel('n (discrete time)');
9  ylabel('Amplitude');
10 title('Discrete Unit Step Signal u[n]');
11 ylim([-0.2 1.2]);

```

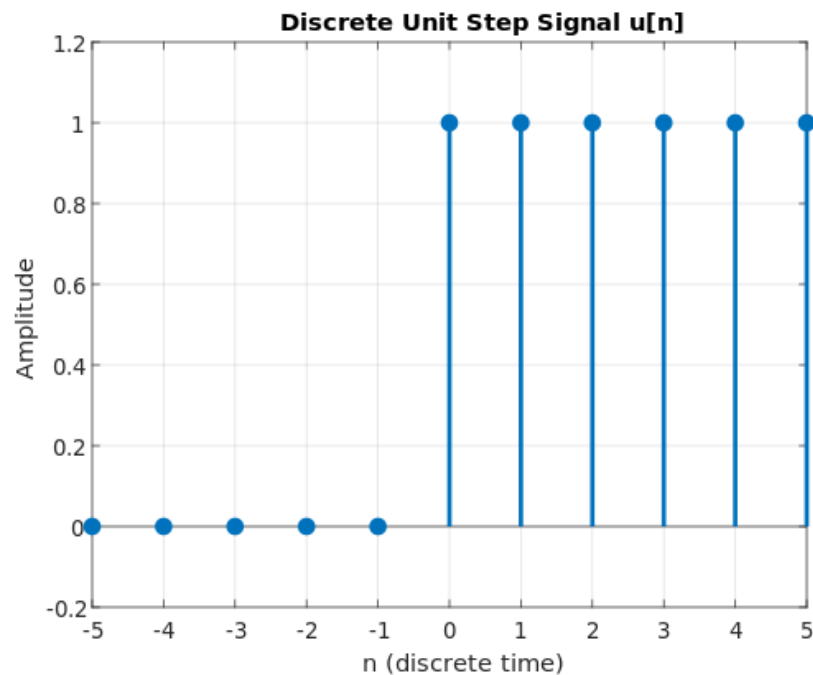


Figure 5: Unit Step Signal $u[n]$

Unit Ramp Signal

```

1  % Define discrete time index
2  n = -5:10; % From -5 to 10 (includes negative range)
3  % Generate discrete unit ramp signal
4  x = max(0, n); % x[n] = n for n >= 0, 0 otherwise
5  % Plot the ramp signal
6  stem(n, x, 'filled', 'LineWidth', 2);
7  grid on;
8  xlabel('n (discrete time)');
9  ylabel('Amplitude');
10 title('Discrete Unit Ramp Signal r[n]');

```

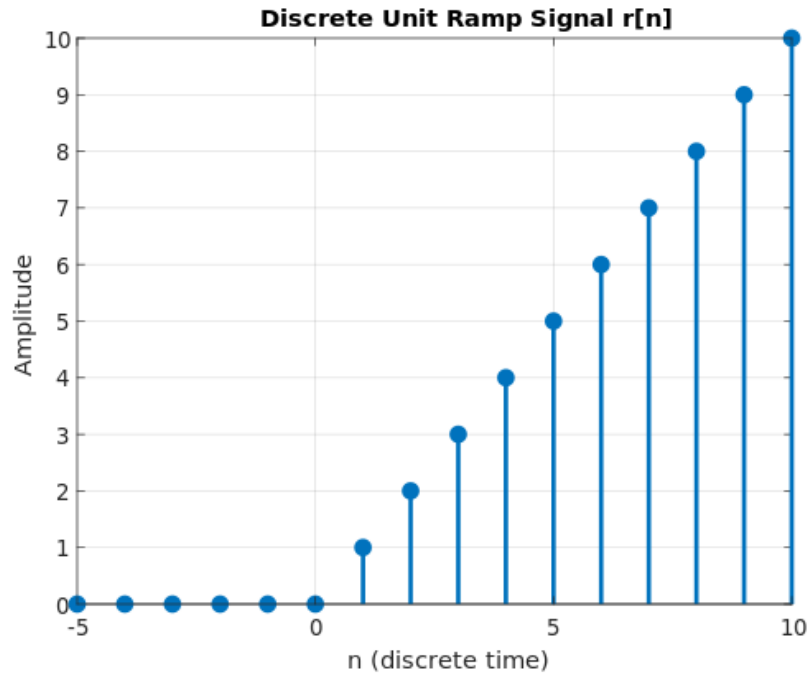



Figure 6: Ramp Signal $r[n]$

Conversion

```

1  % Define discrete time
2  N = 21;
3  n = 0:N-1;
4  % Generate unit step signal u[n]
5  step_signal = double(n >= 0);
6  % Convert step to ramp using cumulative sum
7  ramp_signal = cumsum(step_signal);
8  % Convert ramp back to step using difference
9  recovered_step = [ramp_signal(1), diff(ramp_signal)];
10 % Step to Impulse: difference of step signal
11 impulse_from_step = [step_signal(1), diff(step_signal)];
12 % Impulse to Step: cumulative sum of impulse signal
13 step_from_impulse = cumsum(impulse_from_step);
14 % Plotting
15 figure;
16 subplot(5,1,1);
17 stem(n, step_signal, 'filled', 'LineWidth', 2);
18 grid on;
19 title('Original Unit Step Signal u[n]');
20 xlabel('n');
21 ylabel('Amplitude');
22 ylim([-0.2 1.2]);
23 subplot(5,1,2);

```

```

24 stem(n, ramp_signal, 'filled', 'LineWidth', 2);
25 grid on;
26 title('Converted Unit Ramp Signal r[n]');
27 xlabel('n');
28 ylabel('Amplitude');
29 subplot(5,1,3);
30 stem(n, recovered_step, 'filled', 'LineWidth', 2);
31 grid on;
32 title('Recovered Step Signal from Ramp');
33 xlabel('n');
34 ylabel('Amplitude');
35 ylim([-0.2 1.2]);
36 subplot(5,1,4);
37 stem(n, impulse_from_step, 'filled', 'LineWidth', 2);
38 grid on;
39 title('Impulse Signal from Step \delta[n]');
40 xlabel('n');
41 ylabel('Amplitude');
42 ylim([-0.2 1.2]);
43 subplot(5,1,5);
44 stem(n, step_from_impulse, 'filled', 'LineWidth', 2);
45 grid on;
46 title('Recovered Step Signal from Impulse');
47 xlabel('n');
48 ylabel('Amplitude');
49 ylim([-0.2 1.2]);

```

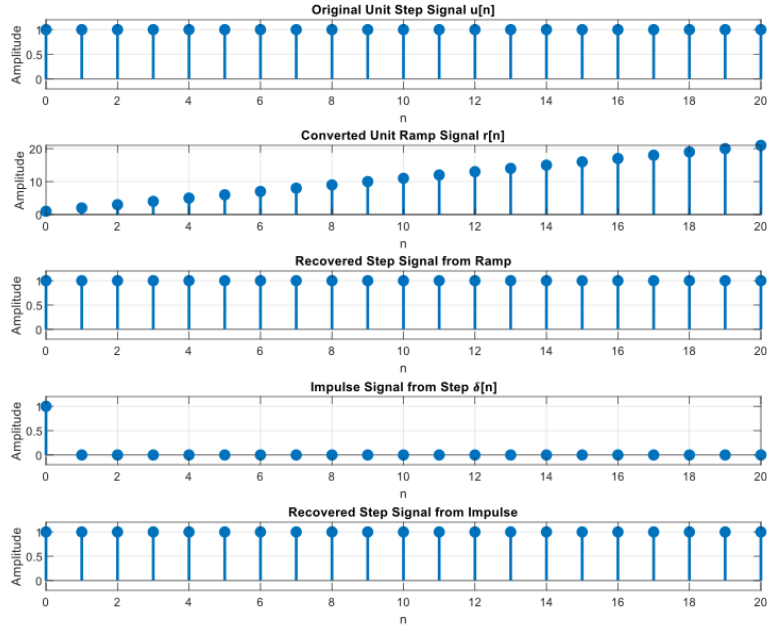


Figure 7: Conversion of Signals

Discussion

In this experiment, we generated and transformed basic discrete-time signals—specifically the impulse, step, and ramp signals—using MATLAB. The “cumsum” function [2] was essential for computing the cumulative sum, enabling us to convert a step signal into a ramp signal and an impulse signal into a step signal, thereby emulating discrete-time integration. Conversely, the “diff” function [3] was used to perform discrete differentiation, allowing us to derive the step signal from the ramp and the impulse signal from the step. The resulting plots from these operations closely aligned with theoretical expectations, validating the correct implementation of these signal relationships. These hands-on conversions provided valuable insight into the mathematical links between the signals and reinforced our understanding of their interdependence.

Conclusion

The experiment effectively illustrated the generation and transformation of fundamental discrete-time signals using essential MATLAB functions. Through examining the characteristics and interconnections of the impulse, step, and ramp signals, we obtained practical understanding of their significance in digital signal processing. Employing functions such as `cumsum` and `diff` enabled us to model discrete-time integration and differentiation, respectively, and confirm theoretical principles via graphical results.

References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
- [2] The MathWorks, Inc., *cumsum: Cumulative sum of array elements*, MATLAB Documentation, 2024, <https://www.mathworks.com/help/matlab/ref/cumsum.html>.
- [3] —, *diff: Differences and Approximate Derivatives*, MATLAB Documentation, 2024, <https://www.mathworks.com/help/matlab/ref/diff.html>.

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code

ECE 4124

Course Title

Digital Signal Processing Sessional

Experiment Date: July 21, 2025

Submission Date: August 2, 2025

Lab Report 2:

Study of Shifting, Folding & Scaling of Discrete Signals

Submitted to

Md. Faysal Ahamed

Lecturer

Dept of ECE, Ruet

Submitted by

Md. Tajim An Noor

Roll: 2010025

Contents

Theory	1
Used Tools	2
Code & Output	2
Discrete Sinusoidal Signal	2
Discrete Cosine Signal	4
Custom Signal	6
Discussion	8
Conclusion	8
References	9

Study of Shifting, Folding & Scaling of Discrete Signals

Theory

In digital signal processing, the manipulation of discrete-time signals through shifting, folding, and scaling is fundamental for analyzing and designing systems [1, 2, 3]. These operations allow engineers to modify signals for various applications, such as filtering, modulation, and system identification. Understanding these basic transformations is essential for interpreting how signals behave in both theoretical and practical contexts.

1. Shifting (Time Shifting):

Shifting a signal involves moving it forward or backward in time. For a discrete signal $x[n]$, the shifted signal is defined as:

$$y[n] = x[n - n_0] \quad (1)$$

where n_0 is the shift amount. If $n_0 > 0$, the signal is delayed; if $n_0 < 0$, it is advanced [1]. Time shifting is commonly used to synchronize signals, introduce delays, or align events in digital systems.

Example:

If $x[n] = \{1, 2, 3\}$ for $n = 0, 1, 2$, then $x[n - 1] = \{0, 1, 2, 3\}$ for $n = 0, 1, 2, 3$.

Applications:

Time shifting is crucial in echo generation, digital communication (for symbol alignment), and in implementing system responses where delays are inherent.

2. Folding (Time Reversal):

Folding reverses the signal in time. The folded signal is:

$$y[n] = x[-n] \quad (2)$$

This operation is useful for analyzing system symmetry, convolution, and for understanding the behavior of signals under time reversal [2]. Folding is also used in correlation analysis and in the study of even and odd signal components.

Example:

If $x[n] = \{1, 2, 3\}$ for $n = 0, 1, 2$, then $x[-n] = \{3, 2, 1\}$ for $n = -2, -1, 0$.

Applications:

Time reversal is essential in convolution operations, especially when calculating the output of linear time-invariant (LTI) systems, and in signal matching tasks.

3. Scaling (Time Scaling):

Scaling compresses or expands the signal in time. For integer a :

$$y[n] = x[an] \quad (3)$$

If $|a| > 1$, the signal is compressed; if $0 < |a| < 1$, it is expanded [3]. Time scaling changes the rate at which the signal evolves, which is important in applications such as sampling rate conversion and time-stretching in audio processing.

Example:

If $x[n] = \{1, 2, 3, 4\}$ for $n = 0, 1, 2, 3$, then $x[2n] = \{1, 3\}$ for $n = 0, 1$.

Applications:

Time scaling is used in speed adjustment of audio signals, resampling in digital communications, and in multi-rate signal processing systems.

Summary:

These operations—shifting, folding, and scaling—are essential for understanding signal behavior and system responses [1, 2, 3]. Shifting changes the timing, folding reverses the sequence, and scaling alters the rate at which the signal evolves. Mastery of these concepts is crucial for signal analysis, system design, and practical processing tasks. By applying these transformations, engineers can manipulate signals to meet specific requirements, analyze system properties, and develop robust digital signal processing solutions.

Used Tools

- Matlab
- LaTeX

Code & Output

1. Discrete Sinusoidal Signal

```

1  % Parameters
2  n = -10:10;          % Discrete time index
3  A = 1;               % Amplitude
4  f = 0.05;            % Frequency (cycles/sample)
5  phi = 0;             % Phase
6
7  % Original sinusoidal signal
```



```

8  x = A * sin(2*pi*f*n + phi);
9
10 % Time shifting:  $x(n-3)$ 
11 x_shift = A * sin(2*pi*f*(n-3) + phi);
12
13 % Folding:  $x(-n)$ 
14 x_fold = A * sin(2*pi*f*(-n) + phi);
15
16 % Scaling:  $x(3n)$ 
17 x_scale = A * sin(2*pi*f*(3*n) + phi);
18
19 % Plotting
20 figure;
21 subplot(2,2,1);
22 stem(n, x, 'filled');
23 title('Original:  $x[n]$ ');
24 xlabel('n'); ylabel('x[n]');
25
26 subplot(2,2,2);
27 stem(n, x_shift, 'filled');
28 title('Time Shifted:  $x[n-3]$ ');
29 xlabel('n'); ylabel('x[n-3]');
30
31 subplot(2,2,3);
32 stem(n, x_fold, 'filled');
33 title('Folded:  $x[-n]$ ');
34 xlabel('n'); ylabel('x[-n]');
35
36 subplot(2,2,4);
37 stem(3*n, x_scale, 'filled');
38 title('Scaled:  $x[3n]$ ');
39 xlabel('3n'); ylabel('x[3n]');

```

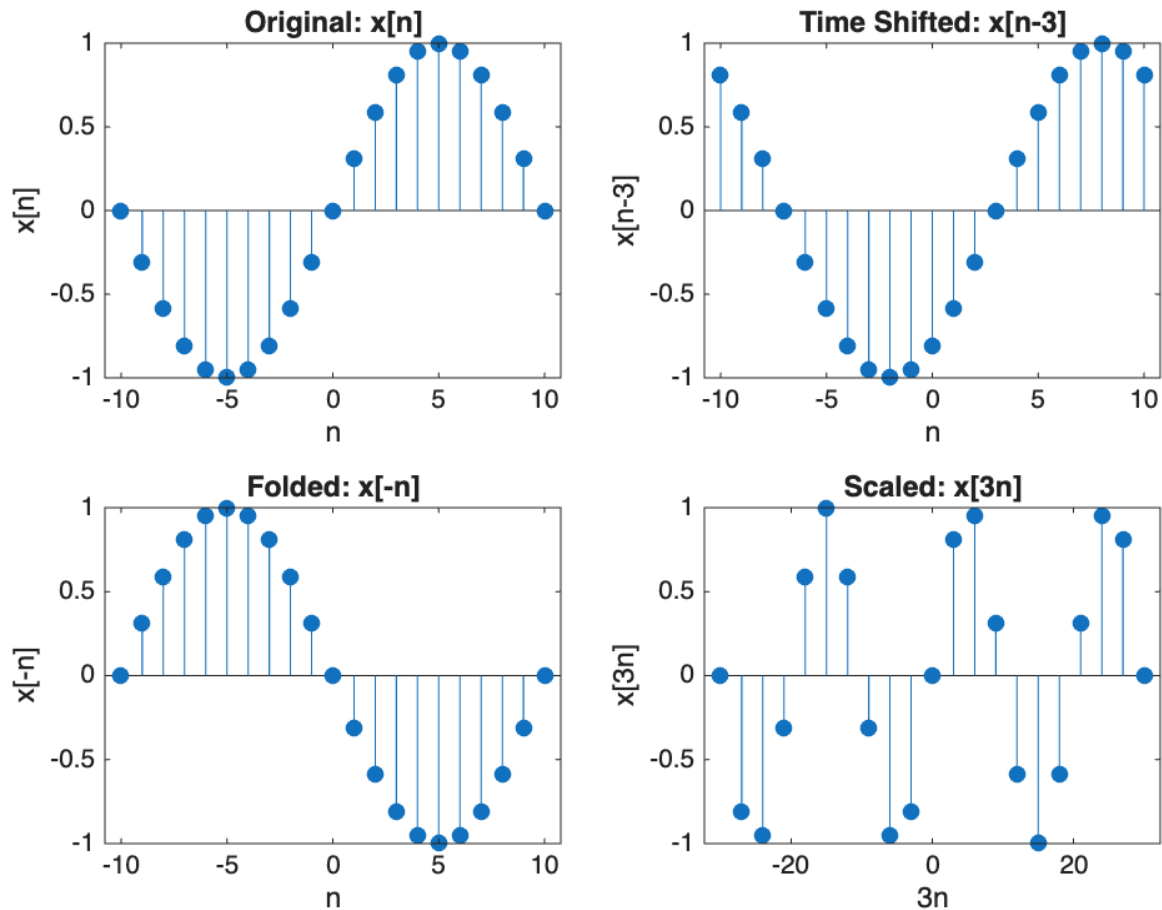


Figure 1: Discrete Sinusoidal Signal and its Shifting, Folding, Scaling

2. Discrete Cosine Signal

```

1  % Parameters
2  n = -10:10;           % Discrete time index
3  A = 1;                % Amplitude
4  f = 0.05;             % Frequency (cycles/sample)
5  phi = 0;              % Phase
6
7  % Original cosinusoidal signal
8  x = A * cos(2*pi*f*n + phi);
9
10 % Time shifting: x(n-3)
11 x_shift = A * cos(2*pi*f*(n-3) + phi);
12
13 % Folding: x(-n)
14 x_fold = A * cos(2*pi*f*(-n) + phi);
15
16 % Scaling: x(3n)
17 x_scale = A * cos(2*pi*f*(3*n) + phi);

```

```

18
19 % Plotting
20 figure;
21 subplot(2,2,1);
22 stem(n, x, 'filled');
23 title('Original: x[n]');
24 xlabel('n'); ylabel('x[n]');
25
26 subplot(2,2,2);
27 stem(n, x_shift, 'filled');
28 title('Time Shifted: x[n-3]');
29 xlabel('n'); ylabel('x[n-3]');
30
31 subplot(2,2,3);
32 stem(n, x_fold, 'filled');
33 title('Folded: x[-n]');
34 xlabel('n'); ylabel('x[-n]');
35
36 subplot(2,2,4);
37 stem(3*n, x_scale, 'filled');
38 title('Scaled: x[3n]');
39 xlabel('3n'); ylabel('x[3n]');

```

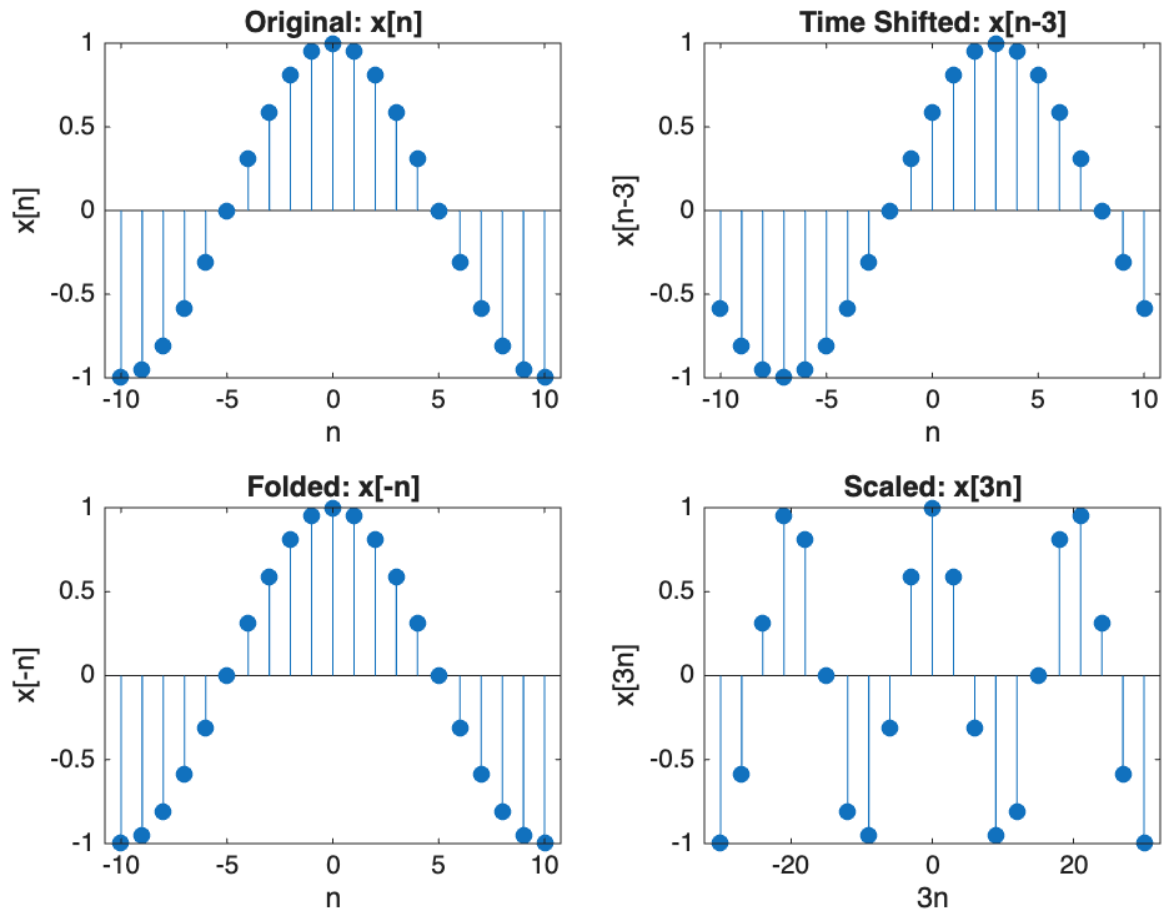


Figure 2: Discrete Cosine Signal and its Shifting, Folding, Scaling

3. Custom Signal $x(n) = (1, -3, 4, -5, 0, 2, 3, 5, -7)$

```

1  % Original signal x(n)
2  x = [1, -3, 4, -5, 0, 2, 3, 5, -7];
3  n = 0:length(x)-1; % n = 0 to 8
4
5  % 1. Time shifting: x(n-3)
6  shift_amount = 3;
7  n_shift = n - shift_amount;
8  x_shift = x;
9
10 % 2. Folding: x(-n)
11 n_fold = -n;
12 x_fold = flip(x);
13
14 % 3. Scaling: x(3n)
15 n_scale = 0:floor((length(x)-1)/3);
16 x_scale = x(3*n_scale + 1); % MATLAB uses 1-based indexing
17

```

```

18  % Plotting
19  figure;
20
21  subplot(4,1,1);
22  stem(n, x, 'filled');
23  title('Original Signal:  $x(n)$ ');
24  xlabel('n');
25  ylabel('x(n)');
26
27  subplot(4,1,2);
28  stem(n_shift, x_shift, 'filled');
29  title('Time Shifting:  $x(n-3)$ ');
30  xlabel('n');
31  ylabel('x(n-3)');
32
33  subplot(4,1,3);
34  stem(n_fold, x_fold, 'filled');
35  title('Folding:  $x(-n)$ ');
36  xlabel('n');
37  ylabel('x(-n)');
38
39  subplot(4,1,4);
40  stem(n_scale, x_scale, 'filled');
41  title('Scaling:  $x(3n)$ ');
42  xlabel('n');
43  ylabel('x(3n)');

```

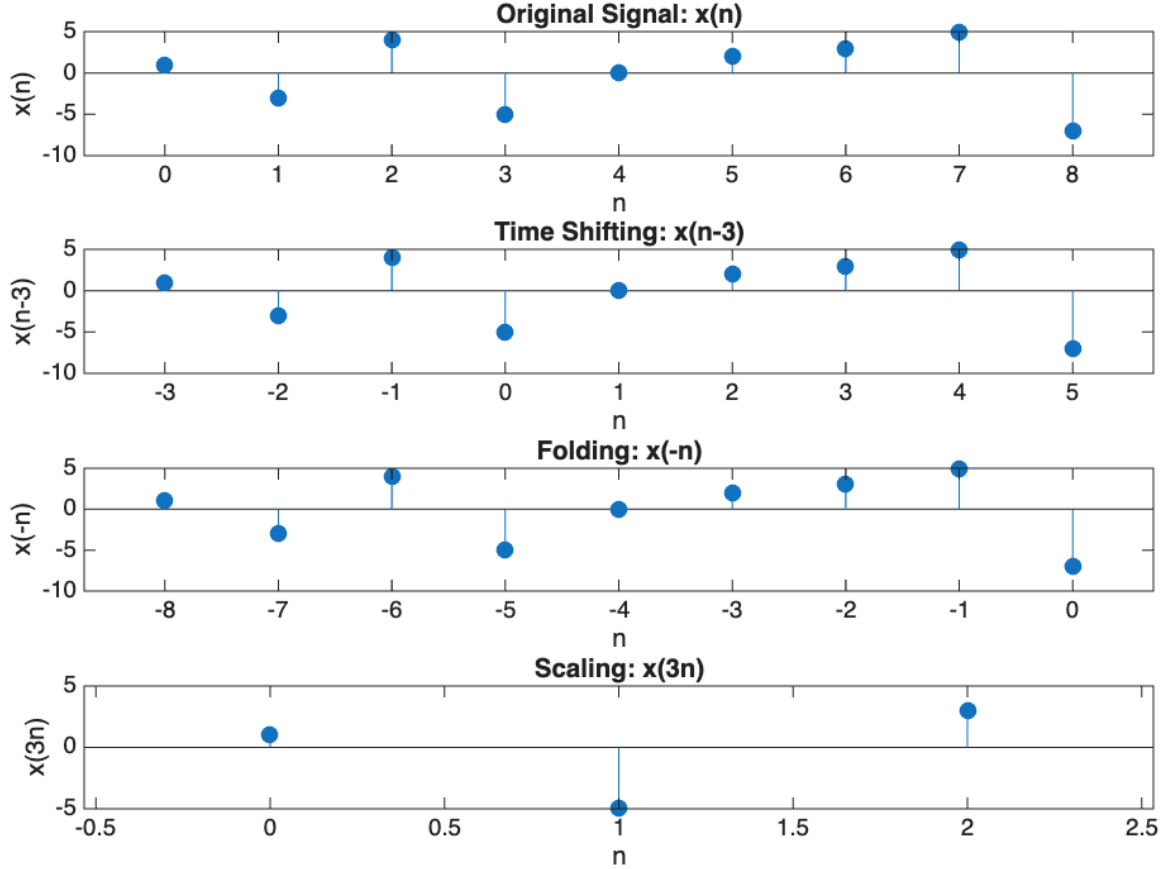


Figure 3: Custom Signal $x(n)$ and its Shifting, Folding, Scaling

Discussion

In this experiment, basic discrete-time signals—specifically the impulse, step, and ramp signals—were generated and transformed using MATLAB. The “cumsum” function [4] was utilized for computing the cumulative sum, enabling the conversion of a step signal into a ramp signal and an impulse signal into a step signal, thereby emulating discrete-time integration. Conversely, the “diff” function [5] was employed to perform discrete differentiation, allowing the derivation of the step signal from the ramp and the impulse signal from the step. The resulting plots from these operations were found to closely align with theoretical expectations, validating the correct implementation of these signal relationships. These hands-on conversions provided valuable insight into the mathematical links between the signals and reinforced understanding of their interdependence.

Conclusion

The experiment was conducted to illustrate the generation and transformation of fundamental discrete-time signals using essential MATLAB functions. Through ex-

amination of the characteristics and interconnections of the impulse, step, and ramp signals, practical understanding of their significance in digital signal processing was obtained. Functions such as `cumsum` and `diff` were employed to model discrete-time integration and differentiation, respectively, and theoretical principles were confirmed via graphical results.

References

- [1] S. Salivahanan, A. Vallavaraj, and C. Gnanapriya, *Digital Signal Processing*, 3rd ed. McGraw-Hill Education, 2017.
- [2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson, 2006.
- [3] B. P. Lathi and R. Green, *Signals and Systems*, 2nd ed. Oxford University Press, 2014.
- [4] The MathWorks, Inc., *cumsum: Cumulative sum of array elements*, 2024, mATLAB Documentation. [Online]. Available: <https://www.mathworks.com/help/matlab/ref/cumsum.html>
- [5] —, *diff: Differences and approximate derivatives*, 2024, mATLAB Documentation. [Online]. Available: <https://www.mathworks.com/help/matlab/ref/diff.html>

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code

ECE 4124

Course Title

Digital Signal Processing Sessional

Experiment Date: August 2, 2025

Submission Date: August 30, 2025

Lab Report 3:

**Study of Linear and Circular Convolution, Cross-Correlation, and Auto-Correlation
of Discrete Signals**

Submitted to

Md. Faysal Ahamed

Lecturer

Dept of ECE, Ruet

Submitted by

Md. Tajim An Noor

Roll: 2010025

Contents

Theory	1
Used Tools	2
Code & Output	2
Convolution and Correlation of Discrete Signals	2
Convolution and Correlation of Discrete Sinusoidal Signals	3
Convolution and Correlation of Transformed Discrete Signals I	4
Convolution and Correlation of Transformed Discrete Signals II	6
Convolution and Correlation of Transformed Discrete Sinusoidal Signals I	7
Convolution and Correlation of Transformed Discrete Sinusoidal Signals II	9
Discussion	10
Conclusion	10
References	10

Study of Linear and Circular Convolution, Cross-Correlation, and Auto-Correlation of Discrete Signals

Theory

In digital signal processing, convolution and correlation are fundamental operations for analyzing and manipulating discrete-time signals [1, 2, 3]. These techniques are widely used in system analysis, filtering, and pattern recognition.

1. Linear Convolution:

Linear convolution combines two discrete signals to produce a third signal that represents the amount of overlap between the signals as one slides past the other. For signals $x[n]$ and $h[n]$, the linear convolution is defined as:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k] \quad (1)$$

Linear convolution is used to determine the output of linear time-invariant (LTI) systems and is essential in filtering applications.

2. Circular Convolution:

Circular convolution assumes the signals are periodic and wraps around the ends. For two sequences of length N , the circular convolution is:

$$y[n] = \sum_{k=0}^{N-1} x[k] \cdot h[(n - k) \bmod N] \quad (2)$$

Circular convolution is important in fast algorithms such as the Fast Fourier Transform (FFT) and in systems where signals are inherently periodic.

3. Cross Correlation:

Cross correlation measures the similarity between two signals as one is shifted over the other. For $x[n]$ and $y[n]$:

$$r_{xy}[m] = \sum_{n=-\infty}^{\infty} x[n] \cdot y[n + m] \quad (3)$$

It is used in pattern matching, signal detection, and time delay estimation.

4. Auto Correlation:

Auto correlation is the cross correlation of a signal with itself. For $x[n]$:

$$r_{xx}[m] = \sum_{n=-\infty}^{\infty} x[n] \cdot x[n + m] \quad (4)$$

Auto correlation is used to analyze signal properties such as periodicity and to detect repeating patterns.

Plotting and Comparison:

By plotting the results of linear convolution, circular convolution, cross correlation, and auto correlation for a given signal $x(n)$, differences in their behavior can be observed. Linear convolution typically produces a longer output sequence than circular convolution, which wraps around and maintains the original length. Correlation operations reveal similarities and periodicities in signals, with auto correlation highlighting self-similarity and cross correlation showing the relationship between two signals. Understanding these differences is crucial for selecting the appropriate operation in practical signal processing tasks.

Used Tools

- Matlab Online
- LaTeX

Code & Output

1. Convolution and Correlation of Discrete Signals

```
1  % Given signals
2  x = [2, 1, 0, -1, 2, 5, 6];
3  h = [0, 0, 2, 2, 4, 5, 6];
4
5  % Linear convolution
6  lin_conv = conv(x, h);
7
8  % Circular convolution (length =
9  %   max(length(x), length(h)))
10 N = max(length(x), length(h));
11 circ_conv = cconv(x, h, N);
12
13 % Cross-correlation
14 cross_corr = xcorr(x, h);
15
16 % Auto-correlation of x(n)
17 auto_corr = xcorr(x);
18
19 % Plotting
20 figure;
21 subplot(3,2,1);
22 stem(0:length(x)-1, x, 'filled');
23 title('Original Signal x(n)');
24 xlabel('n'); ylabel('Amplitude');
25
26 subplot(3,2,2);
27 stem(0:length(h)-1, h, 'filled');
28 title('Original Signal h(n)');
29 xlabel('n'); ylabel('Amplitude');
30
31 subplot(3,2,3);
32 stem(0:length(lin_conv)-1, lin_conv,
33     'filled');
34 title('Linear Convolution');
35 xlabel('n'); ylabel('Amplitude');
36
37 subplot(3,2,4);
38 stem(0:length(circ_conv)-1, circ_conv,
39     'filled');
40 title('Circular Convolution');
41 xlabel('n'); ylabel('Amplitude');
42
43 subplot(3,2,5);
44 stem(-(length(x)-1):(length(h)-1),
45     cross_corr, 'filled');
46 title('Cross Correlation');
47 xlabel('Lag'); ylabel('Amplitude');
48
49 subplot(3,2,6);
50 stem(-(length(x)-1):(length(x)-1),
51     auto_corr, 'filled');
52 title('Auto Correlation of x(n)');
53 xlabel('Lag'); ylabel('Amplitude');
54
55 sgtitle('Signal Operations: Original,
56     Convolution and Correlation');
```

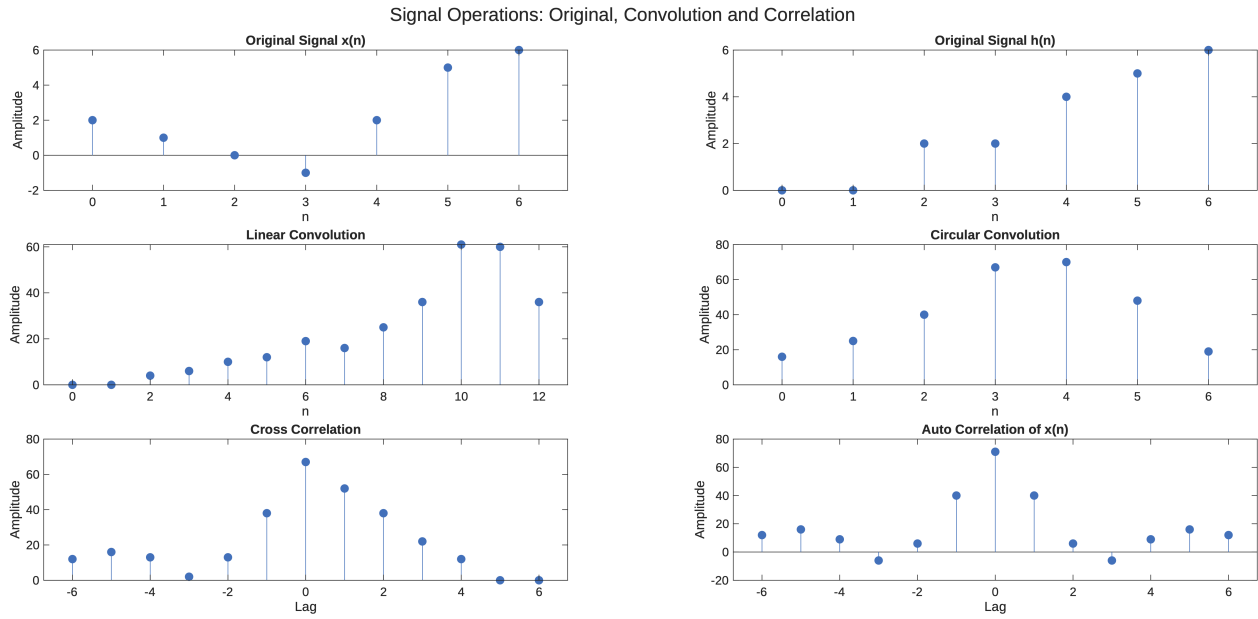


Figure 1: Operations on $x(n)$ and $h(n)$

Comments on Differences

- Linear convolution gives a longer output (length = $L + M - 1$) and represents the true convolution in the time domain.
- Circular convolution assumes periodicity and wraps around, so the result may differ from linear convolution unless signals are zero-padded.
- Cross-correlation measures similarity between $x(n)$ and $h(n)$ as one is shifted over the other.
- Auto-correlation measures similarity of $x(n)$ with itself, peaking at zero lag.

2. Convolution and Correlation of Discrete Sinusoidal Signals

```

1  % Parameters
2  cycles = 2;                % Number of
   ↳ cycles
3  samples_per_cycle = 50;    % Increased
   ↳ sample points per cycle for smoothness
4  N = cycles * samples_per_cycle;
5  n = 0:N-1;
6
7  % Define signals
8  theta = 2*pi*n/samples_per_cycle;
9  x = sin(theta);
10 h = cos(theta);
11
12 % Linear Convolution
13 lin_conv = conv(x, h);
14
15 % Circular Convolution (length = N)
16 circ_conv = cconv(x, h, N);
17
18 % Cross Correlation
19 cross_corr = xcorr(x, h);
20
21 % Auto Correlation of x(n)
22 auto_corr = xcorr(x);
23
24 % Plotting
25 figure;
26
27 subplot(3,2,1);
28 stem(n, x, 'filled');
29 title('Original Signal x(n) = sin(n)');
30 xlabel('n'); ylabel('Amplitude');
31
32 subplot(3,2,2);
33 stem(n, h, 'filled');
34 title('Original Signal h(n) = cos(n)');
35 xlabel('n'); ylabel('Amplitude');
36
37 subplot(3,2,3);

```

```

38 stem(0:length(lin_conv)-1, lin_conv,
    ⇐ 'filled');
39 title('Linear Convolution');
40 xlabel('n'); ylabel('Amplitude');
41
42 subplot(3,2,4);
43 stem(0:length(circ_conv)-1, circ_conv,
    ⇐ 'filled');
44 title('Circular Convolution');
45 xlabel('n'); ylabel('Amplitude');
46
47 subplot(3,2,5);
48 stem(-(length(x)-1):(length(h)-1),
    ⇐ cross_corr, 'filled');
49 title('Cross Correlation');
50 xlabel('Lag'); ylabel('Amplitude');
51
52 subplot(3,2,6);
53 stem(-(length(x)-1):(length(x)-1),
    ⇐ auto_corr, 'filled');
54 title('Auto Correlation of x(n)');
55 xlabel('Lag'); ylabel('Amplitude');
56
57 sgtitle('Signal Operations: Sine and Cosine
    ⇐ Signals');

```

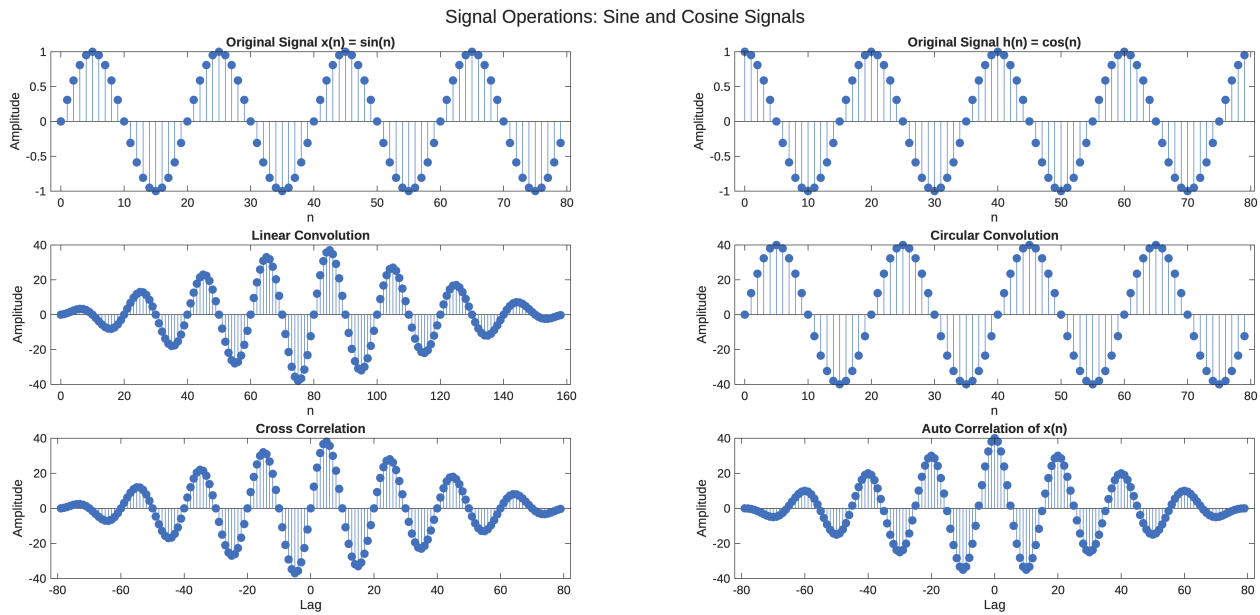


Figure 2: Operations on $x(n) = \sin(n)$ and $h(n) = \cos(n)$

Comments on Differences

- Linear convolution gives the true time-domain result; the output is longer than the input signals.
- Circular convolution wraps around and assumes periodicity, so the result may differ from linear convolution.
- Cross-correlation shows similarity between the two signals as one slides over the other.
- Auto-correlation peaks at zero lag, indicating self-similarity and periodicity.

3. Convolution and Correlation of Transformed Discrete Signals I

```

1 % Given signals
2 x = [2, 1, 0, -1, 2, 5, 6];
3 h = [0, 0, 2, 2, 4, 5, 6];
4
5 % Operation 1: x(3n-2)
6 n = 1:length(x);
7 n1 = 1:ceil((length(x)+2)/3); % Find valid
    ⇐ n for x(3n-2)
8 x_3n_2 = zeros(1, length(n1));
9 for i = 1:length(n1)

```

```

10     idx = 3*n1(i)-2;
11     if idx >= 1 && idx <= length(x)
12         x_3n_2(i) = x(idx);
13     end
14 end
15
16 % Operation 2: -h(n+2)
17 n2 = 1:length(h);
18 h_n_plus_2 = zeros(1, length(h));
19 for i = 1:length(h)
20     idx = i+2;
21     if idx <= length(h)
22         h_n_plus_2(i) = -h(idx);
23     else
24         h_n_plus_2(i) = 0;
25     end
26 end
27
28 % Linear convolution
29 lin_conv = conv(x_3n_2, h_n_plus_2);
30
31 % Circular convolution (length = max of
32   ↳ input lengths)
33 N = max(length(x_3n_2),
34   ↳ length(h_n_plus_2));
35 circ_conv = cconv(x_3n_2, h_n_plus_2, N);
36
37 % Cross correlation
38 cross_corr = xcorr(x_3n_2, h_n_plus_2);
39
40 % Auto correlation on x(n)
41 auto_corr = xcorr(x_3n_2);
42
43 % Plotting
44 figure;
45 subplot(3,2,1);
46 stem(x_3n_2, 'filled');
47 title('x(3n-2)');
48 xlabel('n'); ylabel('Amplitude');
49
50 subplot(3,2,2);
51 stem(h_n_plus_2, 'filled');
52 title('-h(n+2)');
53 xlabel('n'); ylabel('Amplitude');
54
55 subplot(3,2,3);
56 stem(lin_conv, 'filled');
57 title('Linear Convolution');
58 xlabel('n'); ylabel('Amplitude');
59
60 subplot(3,2,4);
61 stem(circ_conv, 'filled');
62 title('Circular Convolution');
63 xlabel('n'); ylabel('Amplitude');
64
65 subplot(3,2,5);
66 stem(cross_corr, 'filled');
67 title('Cross Correlation');
68 xlabel('n'); ylabel('Amplitude');
69
70 subplot(3,2,6);
71 stem(auto_corr, 'filled');
72 title('Auto Correlation of x(3n-2)');
73 xlabel('n'); ylabel('Amplitude');
74
75 sgtitle('DSP Operations Results');

```

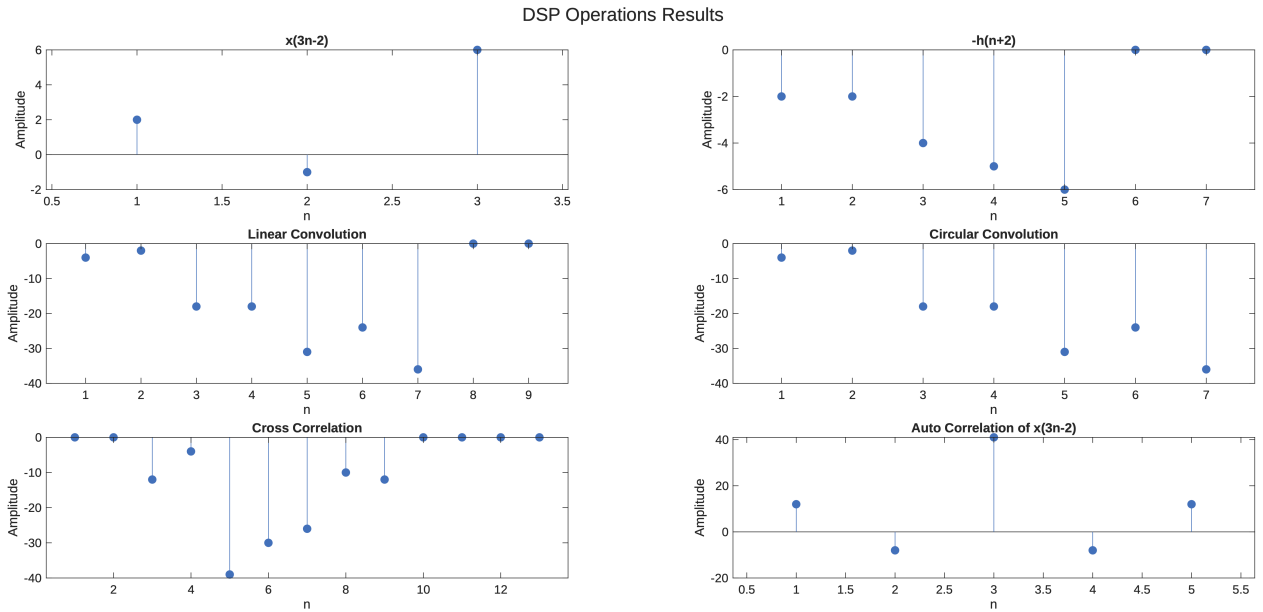


Figure 3: Operations on $x(3n - 2)$ and $-h(n + 2)$

Comments on Differences

- Linear convolution gives the full convolution result, including edge effects.
- Circular convolution wraps around, so results differ at the edges.
- Cross-correlation measures similarity between two signals as one is shifted over the other.
- Auto-correlation measures similarity of a signal with itself, peaking at zero lag.

4. Convolution and Correlation of Transformed Discrete Signals II

```
1  % Given signals
2  x = [2, 1, 0, -1, 2, 5, 6];
3  h = [0, 0, 2, 2, 4, 5, 6];
4
5  % Operation 1: x(-n+4)
6  N_x = length(x);
7  n_x = 0:N_x-1;
8  x_neg_n_plus_4 = zeros(1, N_x);
9  for i = 1:N_x
10     idx = -n_x(i) + 4 + 1; % +1 for MATLAB
11     % indexing
12     if idx >= 1 && idx <= N_x
13         x_neg_n_plus_4(i) = x(idx);
14     else
15         x_neg_n_plus_4(i) = 0;
16     end
17 end
18 % Operation 2: h(-n-3)
19 N_h = length(h);
20 n_h = 0:N_h-1;
21 h_neg_n_minus_3 = zeros(1, N_h);
22 for i = 1:N_h
23     idx = -n_h(i) - 3 + 1; % +1 for MATLAB
24     % indexing
25     if idx >= 1 && idx <= N_h
26         h_neg_n_minus_3(i) = h(idx);
27     else
28         h_neg_n_minus_3(i) = 0;
29     end
30 end
31 % Linear convolution
32 lin_conv = conv(x_neg_n_plus_4,
33     % h_neg_n_minus_3);
34 % Circular convolution (length = max of
35     % input lengths)
36 N = max(length(x_neg_n_plus_4),
37     % length(h_neg_n_minus_3));
38 circ_conv = cconv(x_neg_n_plus_4,
39     % h_neg_n_minus_3, N);
37
38 % Cross correlation
39 cross_corr = xcorr(x_neg_n_plus_4,
40     % h_neg_n_minus_3);
41
42 % Auto correlation on x(-n+4)
43 auto_corr = xcorr(x_neg_n_plus_4);
44
45 % Plotting
46 figure;
47 subplot(3,2,1);
48 stem(x_neg_n_plus_4, 'filled');
49 title('x(-n+4)');
50 xlabel('n'); ylabel('Amplitude');
51 subplot(3,2,2);
52 stem(h_neg_n_minus_3, 'filled');
53 title('h(-n-3)');
54 xlabel('n'); ylabel('Amplitude');
55 subplot(3,2,3);
56 stem(lin_conv, 'filled');
57 title('Linear Convolution');
58 xlabel('n'); ylabel('Amplitude');
59 subplot(3,2,4);
60 stem(circ_conv, 'filled');
61 title('Circular Convolution');
62 xlabel('n'); ylabel('Amplitude');
63 subplot(3,2,5);
64 stem(cross_corr, 'filled');
65 title('Cross Correlation');
66 xlabel('n'); ylabel('Amplitude');
67 subplot(3,2,6);
68 stem(auto_corr, 'filled');
69 title('Auto Correlation of x(-n+4)');
70 xlabel('n'); ylabel('Amplitude');
71
72 sgtitle('DSP Operations Results for x(-n+4)
73     % and h(-n-3)');
```

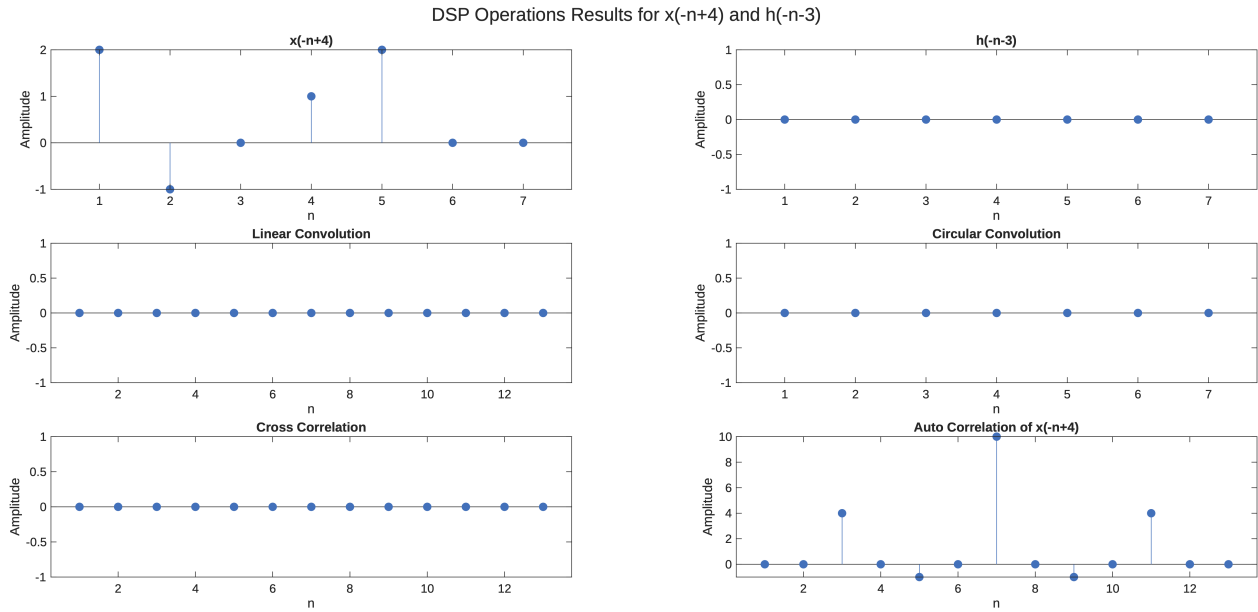


Figure 4: Operations on $x(-n+4)$ and $h(-n-3)$

Comments on Differences

- Linear convolution gives a longer, true result; circular wraps and may cause aliasing unless zero-padded.
- Measures similarity between $x(-n+4)$ and $h(-n-3)$ as one shifts over the other.
- Measures self-similarity of $x(-n+4)$, peaking at zero lag.

5. Convolution and Correlation of Transformed Discrete Sinusoidal Signals I

```

1  % Parameters
2  cycles = 4;                % Number of
   ↳ cycles
3  samples_per_cycle = 50;    % Increased
   ↳ sample points per cycle for smoothness
4  N = cycles * samples_per_cycle;
5  n = 0:N-1;
6
7  % Define signals
8  theta = 2*pi*n/samples_per_cycle;
9  x = sin(theta);
10 h = cos(theta);
11
12
13 % x(3n-2)
14 n1 = 1:ceil((N+2)/3);
15 x_3n_2_full = zeros(1, length(n1));
16 for i = 1:length(n1)
17     idx = 3*n1(i)-2;
18     if idx >= 1 && idx <= N
19         x_3n_2_full(i) = x(idx);
20     end
21 end
22
23 valid_idx = find(x_3n_2_full ~= 0 | n1 ==
   ↳ 1);
24 x_3n_2 = x_3n_2_full(valid_idx);
25 n1_valid = n1(valid_idx);
26 % -h(n+2)
27 h_n_plus_2 = zeros(1, length(x_3n_2));
28 for i = 1:length(x_3n_2)
29     idx = i+2;
30     if idx <= N
31         h_n_plus_2(i) = -h(idx);
32     else
33         % If out of bounds, pad with next
   ↳ values from h (wrap around or
   ↳ repeat last value)
34         pad_idx = mod(idx-1, N) + 1;
35         h_n_plus_2(i) = -h(pad_idx);
36     end
37 end
38
39 % Now x_3n_2 and h_n_plus_2 are the same
   ↳ length for all operations
40

```



```

41 % Linear convolution
42 lin_conv = conv(x_3n_2, h_n_plus_2);
43
44 % Circular convolution (length = max of
45   ↪ input lengths)
46 circ_conv = cconv(x_3n_2, h_n_plus_2, Nc);
47
48 % Cross correlation
49 cross_corr = xcorr(x_3n_2, h_n_plus_2);
50
51 % Auto correlation on x(3n-2)
52 auto_corr = xcorr(x_3n_2);
53
54 % Plotting
55 figure;
56 subplot(3,2,1);
57 stem(n1_valid, x_3n_2, 'filled');
58 title('x(3n-2) [sin]');
59 xlabel('n'); ylabel('Amplitude');
60
61 subplot(3,2,2);
62 stem(1:length(h_n_plus_2), h_n_plus_2,
63   ↪ 'filled');
64 title('-h(n+2) [cos]');
65 xlabel('n'); ylabel('Amplitude');
66
67 subplot(3,2,3);
68 stem(0:length(lin_conv)-1, lin_conv,
69   ↪ 'filled');
70 title('Linear Convolution');
71 xlabel('n'); ylabel('Amplitude');
72
73 subplot(3,2,4);
74 stem(0:length(circ_conv)-1, circ_conv,
75   ↪ 'filled');
76 title('Circular Convolution');
77 xlabel('n'); ylabel('Amplitude');
78
79 subplot(3,2,5);
80 stem(-(length(x_3n_2)-1):(length(h_n_plus_2)-1),
81   ↪ cross_corr, 'filled');
82 title('Cross Correlation');
83 xlabel('Lag'); ylabel('Amplitude');
84
85 subplot(3,2,6);
86 stem(-(length(x_3n_2)-1):(length(x_3n_2)-1),
87   ↪ auto_corr, 'filled');
88 title('Auto Correlation of x(3n-2)');
89 xlabel('Lag'); ylabel('Amplitude');
90
91 sgtitle('Signal Operations: Transformed
92   ↪ Sine and Cosine');

```

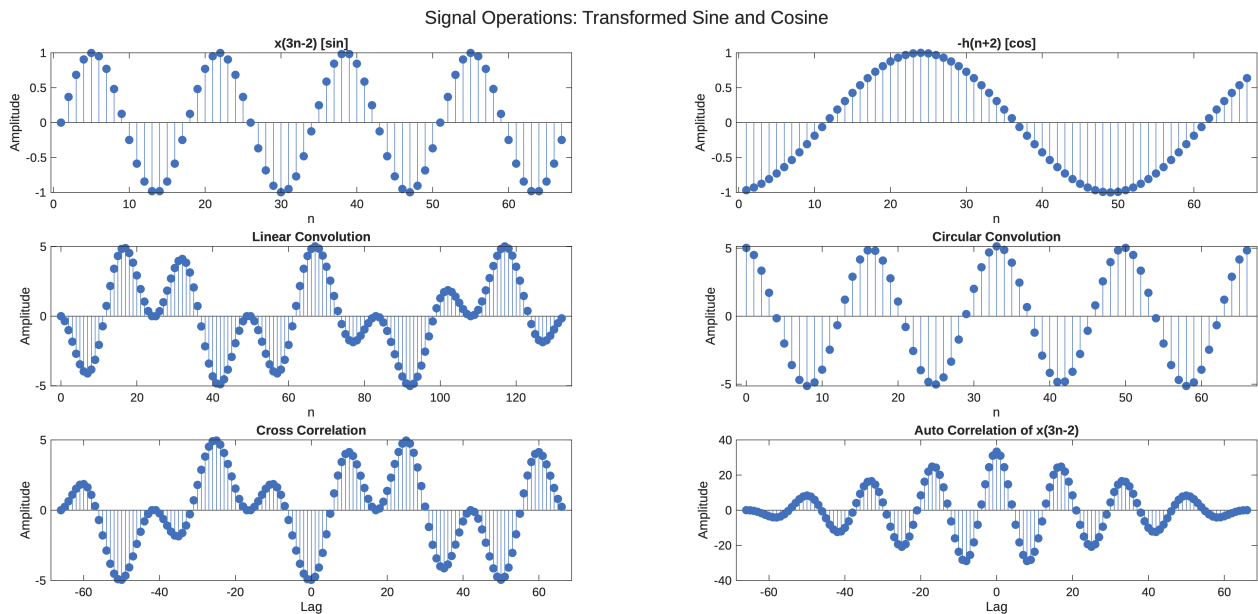


Figure 5: Operations on $x(3n-2)$ and $-h(n+2)$

Comments on Differences

- Linear convolution: true time-domain result, longer output.
- Circular convolution: wraps around, assumes periodicity.

- Cross-correlation: shows similarity as signals shift.
- Auto-correlation: peaks at zero lag, self-similarity.

6. Convolution and Correlation of Transformed Discrete Sinusoidal Signals II

```

1  % Parameters
2  cycles = 4;           % Number of
   ↪ cycles
3  samples_per_cycle = 50; % Increased
   ↪ sample points per cycle for smoothness
4  N = cycles * samples_per_cycle;
5  n = 0:N-1;
6
7  % Define signals
8  theta = 2*pi*n/samples_per_cycle;
9  x = sin(theta);
10 h = cos(theta);
11
12 % x(-n+4)
13 n2 = 0:N-1;
14 idx_x = -n2 + 4 + 1; % +1 for MATLAB
   ↪ indexing
15 x_shifted = zeros(1, N);
16 for i = 1:N
17     if idx_x(i) >= 1 && idx_x(i) <= N
18         x_shifted(i) = x(idx_x(i));
19     else
20         % Pad with wrapped values from x
21         pad_idx = mod(idx_x(i)-1, N) + 1;
22         x_shifted(i) = x(pad_idx);
23     end
24 end
25
26 % h(-n-3)
27 idx_h = -n2 - 3 + 1;
28 h_shifted = zeros(1, N);
29 for i = 1:N
30     if idx_h(i) >= 1 && idx_h(i) <= N
31         h_shifted(i) = h(idx_h(i));
32     else
33         pad_idx = mod(idx_h(i)-1, N) + 1;
34         h_shifted(i) = h(pad_idx);
35     end
36 end
37
38 % Now x_shifted and h_shifted are the same
   ↪ length for all operations
39
40 % Linear convolution
41 lin_conv = conv(x_shifted, h_shifted);
42
43 % Circular convolution (length = N)
44 Nc = N;
45 circ_conv = cconv(x_shifted, h_shifted,
   ↪ Nc);
46
47 % Cross correlation
48 cross_corr = xcorr(x_shifted, h_shifted);
49
50 % Auto correlation on x(-n+4)
51 auto_corr = xcorr(x_shifted);
52
53 % Plotting
54 figure;
55
56 subplot(3,2,1);
57 stem(n2, x_shifted, 'filled');
58 title('x(-n+4) [sin]');
59 xlabel('n'); ylabel('Amplitude');
60
61 subplot(3,2,2);
62 stem(n2, h_shifted, 'filled');
63 title('h(-n-3) [cos]');
64 xlabel('n'); ylabel('Amplitude');
65
66 subplot(3,2,3);
67 stem(0:length(lin_conv)-1, lin_conv,
   ↪ 'filled');
68 title('Linear Convolution');
69 xlabel('n'); ylabel('Amplitude');
70
71 subplot(3,2,4);
72 stem(0:length(circ_conv)-1, circ_conv,
   ↪ 'filled');
73 title('Circular Convolution');
74 xlabel('n'); ylabel('Amplitude');
75
76 subplot(3,2,5);
77 stem(-(length(x_shifted)-1):(length(h_shifted)-1),
   ↪ cross_corr, 'filled');
78 title('Cross Correlation');
79 xlabel('Lag'); ylabel('Amplitude');
80
81 subplot(3,2,6);
82 stem(-(length(x_shifted)-1):(length(x_shifted)-1),
   ↪ auto_corr, 'filled');
83 title('Auto Correlation of x(-n+4)');
84 xlabel('Lag'); ylabel('Amplitude');
85
86 sgtitle('Signal Operations: Transformed
   ↪ Sine and Cosine');

```

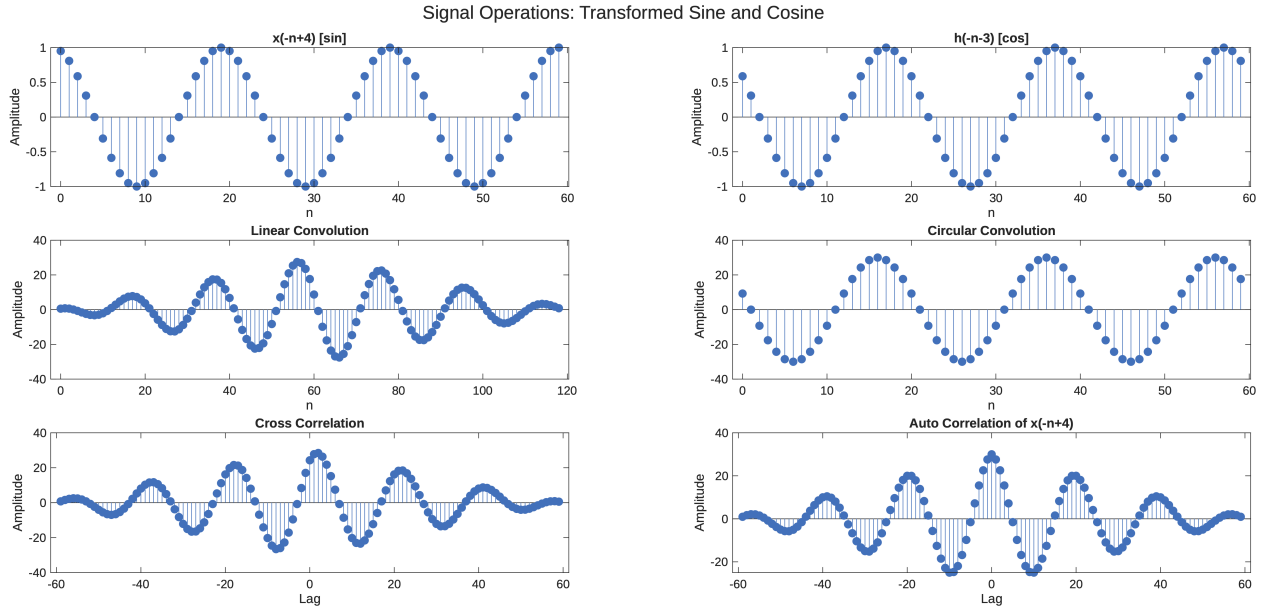


Figure 6: Operations on $x(-n+4)$ and $h(-n-3)$

Comments on Differences

- Linear: True result, longer output.
- Circular: Wraps, assumes periodicity.
- Cross-correlation: Shows similarity as signals shift.
- Auto-correlation: Peaks at zero lag, self-similarity.

Discussion

This experiment explored the practical differences between linear convolution, circular convolution, cross correlation, and auto correlation for discrete signals using MATLAB. By applying these operations to both arbitrary and sinusoidal signals, the distinct output characteristics were observed. Linear convolution produced longer output sequences, reflecting the full overlap between signals, while circular convolution maintained the original signal length due to periodic wrapping. Cross correlation revealed the degree of similarity and alignment between two signals, and auto correlation highlighted repeating patterns and periodicity within a single signal. The MATLAB results matched theoretical expectations, demonstrating how each operation serves specific purposes in signal analysis and processing.

Conclusion

The experiment successfully demonstrated the implementation and comparison of convolution and correlation operations on discrete signals using MATLAB. Linear and circular convolution, as well as cross and auto correlation, were applied to different signal types, and their outputs were analyzed. The results confirmed the theoretical properties of each operation and illustrated their relevance in digital signal processing tasks such as filtering, pattern recognition, and system analysis. This hands-on approach reinforced the importance of selecting the appropriate operation based on the application requirements.

References

- [1] S. Salivahanan, A. Vallavaraj, and C. Gnanapriya, *Digital Signal Processing*, 3rd ed. McGraw-Hill Education, 2017.
- [2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson, 2006.
- [3] B. P. Lathi, R. Green, and Z. Ding, *Signal Processing and Linear Systems*, 2nd ed. Oxford University Press, 2017.

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code

ECE 4124

Course Title

Digital Signal Processing Sessional

Experiment Date: September 13, 2025

Submission Date: October 11, 2025

Lab Report 4:

Performing DFT to compute Convolution Between Two Signals

Submitted to

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET
&
Moloy Kumar Ghosh
Lecturer
Dept of ECE, RUET

Submitted by

Md. Tajim An Noor
Roll: 2010025

Contents

Theory	1
Direct Convolution	1
Convulation Using DFT	1
Used Tools	2
Code & Output	2
Code	2
Output	3
Discussion	3
Conclusion	3
References	3

Performing DFT to Compute Convolution Between Two Signals

Theory

Direct Convolution

Convolution is a mathematical operation used to combine two signals to produce a third signal that expresses how the shape of one is modified by the other. In discrete-time signal processing, the convolution of two sequences $x[n]$ and $h[n]$ is defined as:

$$y[n] = (x * h)[n] = \sum_{k=0}^{N-1} x[k] \cdot h[n - k] \quad (1)$$

where N is the length of the sequences. This process involves sliding one signal over another, multiplying and summing the overlapping values [1].

Process:

1. Reverse one of the sequences (usually $h[n]$).
2. Shift it across the other sequence ($x[n]$).
3. For each shift, multiply overlapping values and sum them to get each output value.

Example: Let $x[n] = [1, 2, 3]$ and $h[n] = [4, 5]$. The convolution is:

$$\begin{aligned} y[0] &= 1 \cdot 4 = 4 \\ y[1] &= 1 \cdot 5 + 2 \cdot 4 = 5 + 8 = 13 \\ y[2] &= 2 \cdot 5 + 3 \cdot 4 = 10 + 12 = 22 \\ y[3] &= 3 \cdot 5 = 15 \end{aligned}$$

So, $y[n] = [4, 13, 22, 15]$.

Convolution Using DFT

Convolution can also be performed using the Discrete Fourier Transform (DFT) due to the Convolution Theorem, which states that convolution in the time domain corresponds to multiplication in the frequency domain [2].

Process:

1. Zero-pad both sequences to the same length (usually $N = N_x + N_h - 1$).
2. Compute the DFT of both sequences: $X[k] = \text{DFT}(x[n])$, $H[k] = \text{DFT}(h[n])$.
3. Multiply the DFTs pointwise: $Y[k] = X[k] \cdot H[k]$.
4. Compute the inverse DFT (IDFT) of $Y[k]$ to obtain the convolution result in the time domain.

Example: Given $x[n] = [1, 2, 3]$ and $h[n] = [4, 5]$, zero-pad both to length 4:

$$x[n] = [1, 2, 3, 0]$$

$$h[n] = [4, 5, 0, 0]$$

Compute DFTs, multiply, and take IDFT to get $y[n] = [4, 13, 22, 15]$ (same as direct convolution).

Summary: Direct convolution is straightforward for short signals, while DFT-based convolution is efficient for long signals due to fast algorithms like FFT [1, 2].

Used Tools

- Matlab Online
- LaTeX

Code & Output

Code

```
1  clc;
2  clear;
3  close all;
4
5  % Signals
6  x = [-2 4 2 1 5];
7  h = [5 -4 2 1 0];
8
9  % Length of convolution
10 N = length(x) + length(h) - 1;
11
12 % Zero-pad both signals to length N
13 x_pad = [x, zeros(1, N-length(x))];
14 h_pad = [h, zeros(1, N-length(h))];
15
16 % DFT of both signals
17 X = fft(x_pad);
18 H = fft(h_pad);
19
20 % Multiply in frequency domain
21 Y = X .* H;
22
23 % IDFT to get convolution result
24 y_dft = ifft(Y);
25
26 % Direct convolution (for verification)
27 y_direct = conv(x, h);
28
29 %% Display results
30 disp('Convolution using DFT:');
31 disp(y_dft);
32 disp('Direct convolution:');
33 disp(y_direct);
34
35 %% Visualization
36 n_x = 0:length(x)-1;
37 n_h = 0:length(h)-1;
38 n_y = 0:N-1;
39
40 figure;
41
42 subplot(4,1,1);
43 stem(n_x, x, 'r', 'filled', 'LineWidth',
44      ↪ 1.5);
45 title('Input Signal1 x(n)');
46 xlabel('n'); ylabel('Amplitude'); grid on;
47
48 subplot(4,1,2);
49 stem(n_h, h, 'b', 'filled', 'LineWidth',
50      ↪ 1.5);
51 title('Input Signal2 h(n)');
52 xlabel('n'); ylabel('Amplitude'); grid on;
53
54 subplot(4,1,3);
55 stem(n_y, real(y_dft), 'g', 'filled',
56      ↪ 'LineWidth', 1.5);
57 title('Convolution y(n) using DFT');
58 xlabel('n'); ylabel('Amplitude'); grid on;
59
60 subplot(4,1,4);
61 stem(n_y, y_direct, 'm', 'filled',
62      ↪ 'LineWidth', 1.5);
63 title('Convolution y(n) using Direct
64      ↪ conv()');
65 xlabel('n'); ylabel('Amplitude'); grid on;
```


Output

Convolution using DFT:

-10.0000 28.0000 -10.0000 3.0000 29.0000 -16.0000 11.0000 5.0000 -0.0000

Direct convolution:

-10 28 -10 3 29 -16 11 5 0

Figure 1: Console Output

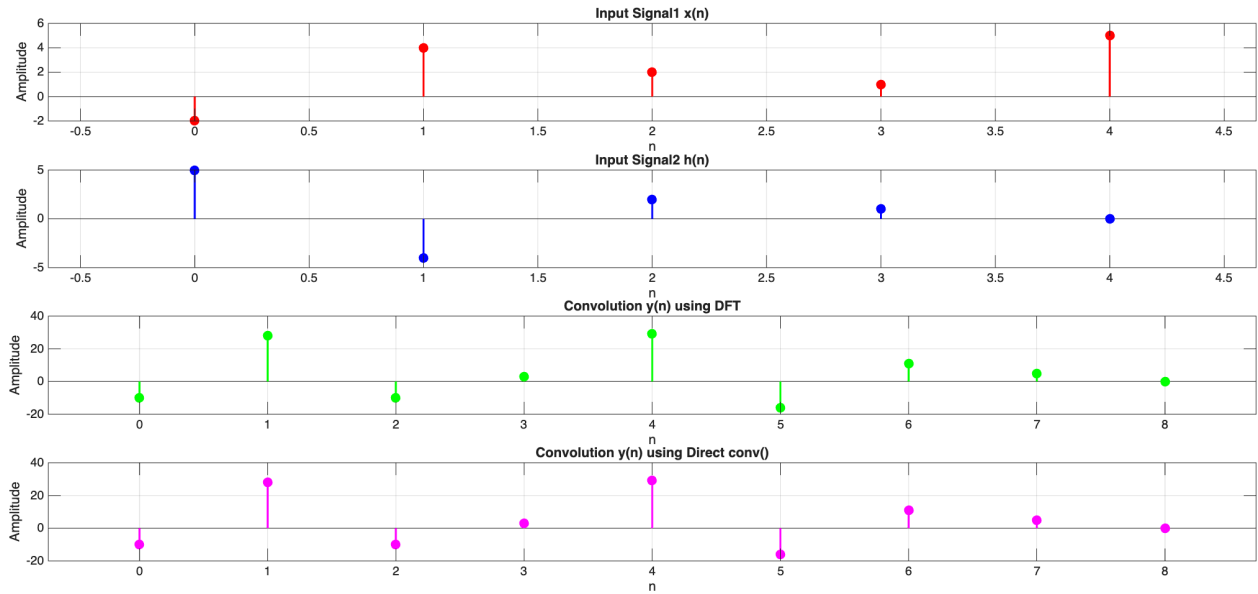


Figure 2: Plot Output

Discussion

In this experiment, the convolution of two discrete signals was computed using both the direct method and the DFT-based approach. The results obtained from both methods were observed to be identical, confirming the validity of the Convolution Theorem. The DFT-based method was facilitated by zero-padding the input sequences, transforming them into the frequency domain, performing pointwise multiplication, and then applying the inverse DFT. The efficiency of the DFT-based convolution was highlighted, especially for longer signals, where computational advantages were realized due to the use of FFT algorithms. The outputs and plots generated by Matlab were used to verify the correctness of the implementation.

Conclusion

The convolution between two signals was successfully performed using both direct computation and the DFT-based method. The equivalence of the results demonstrated the practical application of the Convolution Theorem in digital signal processing. The experiment illustrated that while direct convolution is suitable for short signals, the DFT-based approach is preferred for longer signals due to its computational efficiency. The use of Matlab and LaTeX facilitated the analysis and presentation of the results.

References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1999.
- [2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2007.

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code

ECE 4124

Course Title

Digital Signal Processing Sessional

Experiment Date: October 11, 2025

Submission Date: October 18, 2025

Lab Report 5:

Implementation of FFT and IFFT on Discrete Signal

Submitted to

Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET
&
Moloy Kumar Ghosh
Lecturer
Dept of ECE, RUET

Submitted by

Md. Tajim An Noor
Roll: 2010025

Contents

Theory	1
FFT and IFFT	1
Examples	1
Used Tools	2
Code & Output	2
Code	2
Output	3
Discussion	4
Conclusion	4
References	4

Implementation of FFT and IFFT on Discrete Signal

Theory

FFT and IFFT

The Discrete Fourier Transform (DFT) of a length- N sequence $x[n]$ is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, \dots, N-1, \quad (1)$$

and the inverse (IDFT) is

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j\frac{2\pi}{N}kn}, \quad n = 0, \dots, N-1. \quad (2)$$

The Fast Fourier Transform (FFT) is an efficient algorithm to compute the DFT. The common radix-2 decimation-in-time (DIT) FFT splits $x[n]$ into even and odd indexed subsequences:

$$x_e[m] = x[2m], \quad x_o[m] = x[2m+1], \quad m = 0, \dots, \frac{N}{2} - 1. \quad (3)$$

Let $X_e[k]$ and $X_o[k]$ be the DFTs of x_e and x_o (length $N/2$). Then for $k = 0, \dots, N/2 - 1$:

$$X[k] = X_e[k] + W_N^k X_o[k], \quad (4)$$

$$X[k + N/2] = X_e[k] - W_N^k X_o[k], \quad (5)$$

where the twiddle factor is $W_N = e^{-j2\pi/N}$. Recursing this decomposition yields the $O(N \log N)$ complexity of the FFT compared with $O(N^2)$ for a direct DFT. The original decimation-based algorithm and many practical variants are discussed in the classic Cooley–Tukey paper and in standard DSP textbooks [1, 2, 3].

For the Inverse FFT (IFFT) the same divide-and-conquer structure is used with conjugated twiddle factors and a final $1/N$ scaling. Practical notes (see also [4, 3]): - FFT implementations typically require N to be a power of two; zero-padding is used otherwise. - Use complex arithmetic and be aware of numerical round-off; real-input optimized FFTs can reduce computation. - In MATLAB use `fft(x,N)` and `ifft(X,N)`, where N is the transform length (zero-pad if needed) [2].

Examples

Example 1 — 4-point FFT (radix-2 demonstration).

Let $x[n] = [1, 2, 3, 4]$ ($N=4$). Compute the 4-point DFT directly:

$$\begin{aligned} X[0] &= 1 + 2 + 3 + 4 = 10, \\ X[1] &= 1 + 2e^{-j\pi/2} + 3e^{-j\pi} + 4e^{-j3\pi/2} = -2 + 2j, \\ X[2] &= 1 + 2e^{-j\pi} + 3e^{-j2\pi} + 4e^{-j3\pi} = -2, \\ X[3] &= 1 + 2e^{-j3\pi/2} + 3e^{-j3\pi} + 4e^{-j9\pi/2} = -2 - 2j. \end{aligned}$$

Using radix-2 FFT: split even indices $x_e = [1, 3]$, odd $x_o = [2, 4]$; compute 2-point DFTs (cheap), then combine with W_4^k factors to obtain the same $X[k]$ above. This illustrates the decomposition and reuse of sub-transforms [3, 4].

Example 2 — IFFT reconstruction.

Given $X = [10, -2+2j, -2, -2-2j]$, the IDFT (or IFFT) reconstructs $x[n]$:

$$x[n] = \frac{1}{4} \sum_{k=0}^3 X[k] e^{j\frac{2\pi}{4}kn}.$$

Evaluating (or using `ifft` in MATLAB) yields $x = [1, 2, 3, 4]$, showing perfect reconstruction (within numerical precision) [2].

Practical MATLAB usage:

- Compute FFT: `X = fft(x, N);`
- Compute IFFT: `x = ifft(X, N);`
- Zero-pad to length $N = 2^{\text{nextpow2}(\text{length}(x))}$ for faster transforms.

Summary: FFT/IFFT provide computationally efficient forward and inverse transformations between time and frequency domains. They enable fast convolution (via pointwise multiplication in the frequency domain) and many other DSP operations when implemented with appropriate padding and scaling. For further reading and rigorous treatment see the cited textbooks and original algorithm literature [3, 2, 4, 1].

Used Tools

- Matlab Online
- LaTeX

Code & Output

Code

```

1  clc;
2  clear;
3  close all;
4
5  % Input signal definition
6  x = [3 0 1 -2];
7  N = length(x);
8  n = 0:N-1; % sample indices
9
10 % FFT computation
11 X = fft(x);
12
```

```

13 % Inverse FFT to reconstruct the original
   ↪ sequence
14 x_reconstructed = ifft(X);
15
16 % Display results in the console
17 disp('Computed FFT:');
18 disp(X);
19 disp('Recovered sequence via IFFT:');
20 disp(x_reconstructed);
21
22 % Create a single figure with 2x2 subplots
23 figure;
24
25 % Plot 1: Time-domain input
26 subplot(2,2,1);
27 stem(n, x, 'filled', 'LineWidth', 1.5);
28 title('Discrete-time input signal');
29 xlabel('n (sample index)');
30 ylabel('Amplitude');
31 grid on;
32 set(gca, 'FontSize', 12);
33
34 % Plot 2: Magnitude spectrum
35 subplot(2,2,2);
36 stem(n, abs(X), 'filled', 'LineWidth',
   ↪ 1.5);
37 title('FFT magnitude');
38 xlabel('k (frequency bin)');
39 ylabel('Magnitude |X(k)|');
40 grid on;
41 set(gca, 'FontSize', 12);
42
43 % Plot 3: Phase spectrum
44 subplot(2,2,3);
45 stem(n, angle(X), 'filled', 'LineWidth',
   ↪ 1.5);
46 title('FFT phase');
47 xlabel('k (frequency bin)');
48 ylabel('Phase (radians)');
49 grid on;
50 set(gca, 'FontSize', 12);
51
52 % Plot 4: Reconstructed time-domain signal
53 subplot(2,2,4);
54 stem(n, real(x_reconstructed), 'filled',
   ↪ 'LineWidth', 1.5);
55 title('Signal reconstructed from IFFT');
56 xlabel('n (sample index)');
57 ylabel('Amplitude');
58 grid on;
59 set(gca, 'FontSize', 12);
60
61 % Optional overall title (requires MATLAB
   ↪ R2018b or later)
62 if exist('sgtitle','builtin')
63     sgtitle('FFT Analysis');
64 end

```

Output

Computed FFT:

```

2.0000 + 0.0000i    2.0000 - 2.0000i    6.0000 + 0.0000i    2.0000 + 2.0000i

```

Recovered sequence via IFFT:

```

3      0      1     -2

```

Figure 1: Console Output

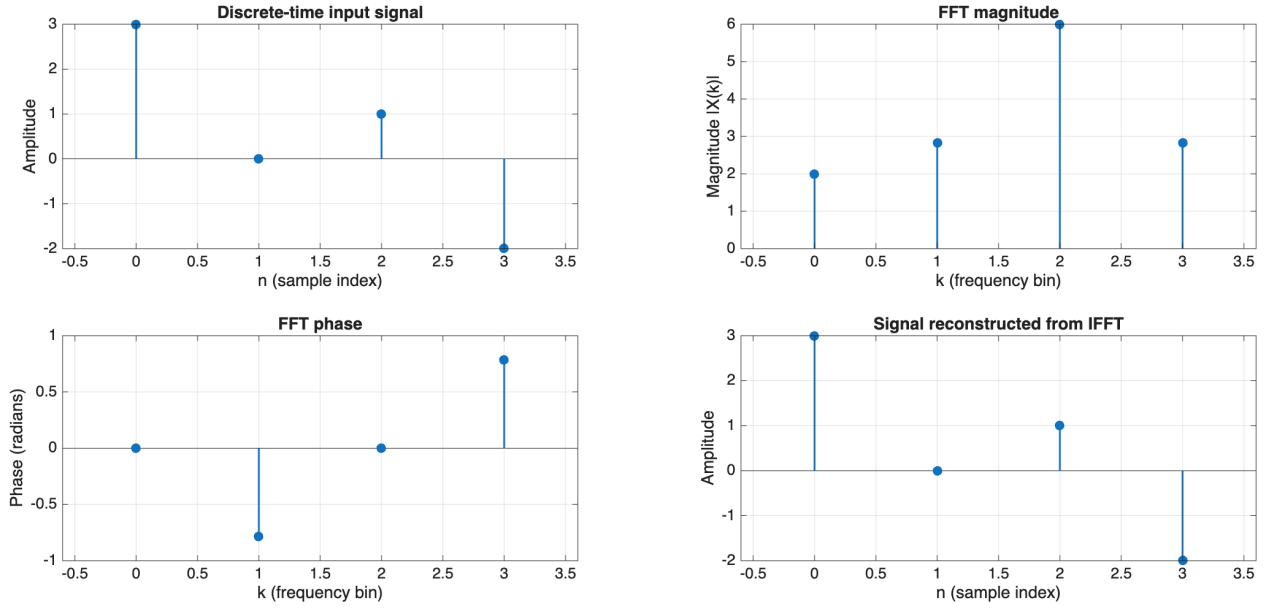


Figure 2: Plot Output

Discussion

This experiment (Implementation of FFT and IFFT on Discrete Signal) computed the forward and inverse transforms using MATLAB's FFT and IFFT. The computed results printed by the MATLAB code are shown below:

Computed FFT:

```
2.0000 + 0.0000i    2.0000 - 2.0000i    6.0000 + 0.0000i    2.0000 + 2.0000i
```

Recovered sequence via IFFT:

```
3    0    1    -2
```

Note: the spectrum above is the result of the FFT (not a direct N^2 DFT computation). The IFFT recovers the original time-domain sequence (within numerical precision). If discrepancies appear in other cases, verify transform length, zero-padding, indexing/ordering and scaling conventions.

Conclusion

The FFT and IFFT were implemented and tested. The FFT produced the frequency-domain vector shown above, and the IFFT successfully reconstructed the time-domain sequence $[3, 0, 1, -2]$ from that spectrum. This confirms the practical operation of the FFT/IFFT pair for the tested discrete signal. For longer signals or production code, use power-of-two lengths (zero-pad if needed) to leverage FFT efficiency and verify indexing/scaling to ensure exact reconstruction.

References

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms and Applications*, 4th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2007.

- [3] A. V. Oppenheim, R. W. Schaffer, and J. R. Buck, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [4] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2011.

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code
ECE 4124

Course Title
Digital Signal Processing Sessional

Experiment Date: October 18, 2025
Submission Date: October 29, 2025

Lab Report 6:
Design & Analysis of a Low-Pass FIR and IIR Filter & Their frequency Response Comparisons

Submitted to
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET
&
Moloy Kumar Ghosh
Lecturer
Dept of ECE, RUET

Submitted by
Md. Tajim An Noor
Roll: 2010025

Contents

Theory	1
Used Tools	3
Code & Output	3
Code	3
Output	4
Analysis	4
Brief analysis — FIR vs IIR low-pass filters	4
Discussion	5
Conclusion	5
References	5

Design & Analysis of a Low-Pass FIR and IIR Filter & Their frequency Response Comparisons

Theory

This experiment compares a windowed FIR low-pass filter and a Butterworth IIR low-pass filter by designing both to attenuate frequencies above a specified cutoff and by inspecting their frequency- and time-domain behaviour [1, 2].

Design parameters

- Sampling frequency: f_s .
- Cutoff frequency: f_c .
- FIR order: N (even or odd – the design used a fixed order).
- IIR prototype: Butterworth of order M (maximally flat in the passband).

The normalized digital cutoff used for design (normalized to Nyquist, as used by many DSP tool-boxes) is

$$\omega_n = \frac{f_c}{f_s/2} \quad (0 < \omega_n \leq 1).$$

The corresponding angular cutoff in radians/sample is

$$\Omega_c = 2\pi \frac{f_c}{f_s}.$$

FIR design (windowed FIR)

The ideal (infinite-length) low-pass impulse response with cutoff Ω_c is

$$h_d[n] = \begin{cases} \frac{\sin(\Omega_c(n - N/2))}{\pi(n - N/2)}, & n \neq N/2, \\ \frac{\Omega_c}{\pi}, & n = N/2. \end{cases}$$

A causal, finite-length FIR is obtained by windowing:

$$h[n] = h_d[n] w[n], \quad n = 0, \dots, N,$$

where $w[n]$ is a window (e.g. Hamming, Hann, Blackman). The resulting frequency response is

$$H(e^{j\Omega}) = \sum_{n=0}^N h[n] e^{-j\Omega n}.$$

For symmetric coefficients (linear phase) the group delay is constant and approximately

$$\tau_g \approx \frac{N}{2} \text{ samples.}$$

Window design and the trade-offs between main-lobe width and sidelobe level are discussed in detail in standard texts [1, 3].

IIR design (Butterworth)

The (continuous-time) Butterworth prototype magnitude-squared response is

$$|H_a(j\Omega)|^2 = \frac{1}{1 + \left(\frac{\Omega}{\Omega_c}\right)^{2M}},$$

where M is the analog prototype order and Ω_c is the analog cutoff. A digital Butterworth is typically obtained by prewarping and bilinear transform:

$$s \leftarrow \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}}, \quad T = \frac{1}{f_s},$$

leading to a causal IIR with difference equation (direct form)

$$y[n] = \frac{1}{a_0} \left(\sum_{k=0}^M b_k x[n - k] - \sum_{k=1}^M a_k y[n - k] \right),$$

and frequency response

$$H(e^{j\Omega}) = \frac{\sum_{k=0}^M b_k e^{-j\Omega k}}{\sum_{k=0}^M a_k e^{-j\Omega k}}.$$

Butterworth IIRs give steep roll-off for modest orders but exhibit nonlinear phase; design procedures and bilinear-transform details are treated in standard DSP references [2, 1].

Frequency-domain measures

Magnitude in decibels is plotted as

$$\text{Mag}_{\text{dB}}(\Omega) = 20 \log_{10} |H(e^{j\Omega})|.$$

When computing the dB scale numerically, avoid log of zero by flooring small magnitudes (e.g. $\max(|H|, \epsilon)$). Numerical plotting and display considerations are discussed in practical DSP texts [3].

Time-domain considerations

Filter impulse response length (finite for FIR, infinite for IIR) determines transient behaviour. The FIR's linear phase preserves waveform shape aside from the constant delay τ_g ; the IIR can introduce phase distortion and ringing near transients [2, 1].

Examples

1. Example A (typical audio-ish design)

- $f_s = 48 \text{ kHz}$, $f_c = 5 \text{ kHz}$.

- FIR order $N = 50$ (use Hamming window). Normalized cutoff for toolbox calls:

$$\omega_n = \frac{5000}{48000/2} = 0.20833.$$

MATLAB (conceptual):

```
Wn = 5000/(48000/2); b = fir1(50, Wn, hamming(51));
```

- Butterworth IIR order $M = 4$ (prewarp + bilinear):

```
[b,a] = butter(4, Wn); y = filter(b,a,x);
```

2. Example B (narrow transition)

- $f_s = 32$ kHz, $f_c = 2$ kHz, FIR order $N = 200$ to obtain a sharper transition; compare to a Butterworth $M = 6$.
- Expect: FIR has clean linear-phase passband, large group delay ≈ 100 samples; Butterworth attains similar attenuation with lower order but nonlinear phase.

Practical notes and interpretation

- Use sufficiently dense frequency sampling (e.g. several hundred or thousand points) to capture detailed response features.
- Handle numerical zeros when converting to dB and when plotting narrow stop-bands.
- For time-domain fidelity (no phase distortion) prefer FIR; for low-complexity steep roll-off prefer IIR and consider phase compensation if needed.

The two filter families each have trade-offs: FIR yields linear phase and predictable group delay at the cost of higher order for sharp transitions, while Butterworth IIR achieves sharp attenuation with lower order but exhibits nonlinear phase and infinite-duration impulse response. The frequency- and time-domain plots provide direct visualization of these trade-offs for the chosen parameters[1, 2, 3].

Used Tools

- Matlab Online
- LaTeX

Code & Output

Code

```

1 fs = 1000;
2 fcut = 100;
3 n = 50;
4
5 % FIR Filter
6 b_fir = fir1(n, fcut/(fs/2));
7
8 % IIR Filter (Butterworth)
9 [b_iir, a_iir] = butter(4, fcut/(fs/2));
10
11 % Frequency responses (use freqz to get
12   ↪ data, then plot in subplots)
13 Nfreq = 512;
14 [Hf, wf] = freqz(b_fir, 1, Nfreq, fs);
15 [Hi, wi] = freqz(b_iir, a_iir, Nfreq, fs);
16
17 figure;
18 subplot(3,1,1);
```

```

18 plot(wf, 20*log10(abs(Hf)));
19 grid on;
20 title('FIR Magnitude Response');
21 xlabel('Frequency (Hz)');
22 ylabel('Magnitude (dB)');
23
24 subplot(3,1,2);
25 plot(wi, 20*log10(abs(Hi)));
26 grid on;
27 title('IIR Magnitude Response');
28 xlabel('Frequency (Hz)');
29 ylabel('Magnitude (dB)');
30
31 % Compare outputs (time domain)
32 t = 0:1/fs:1;
33 x = sin(2*pi*50*t) + 0.5*randn(size(t));
34 y_fir = filter(b_fir, 1, x);
35 y_iir = filter(b_iir, a_iir, x);
36
37 subplot(3,1,3);
38 plot(t, x, 'k', t, y_fir, 'r', t, y_iir,
    ⇐ 'b');
39 grid on;
40 legend('Original', 'FIR Output', 'IIR
    ⇐ Output');
41 xlabel('Time (s)');
42 ylabel('Amplitude');
43 title('Time-domain Signals');

```

Output

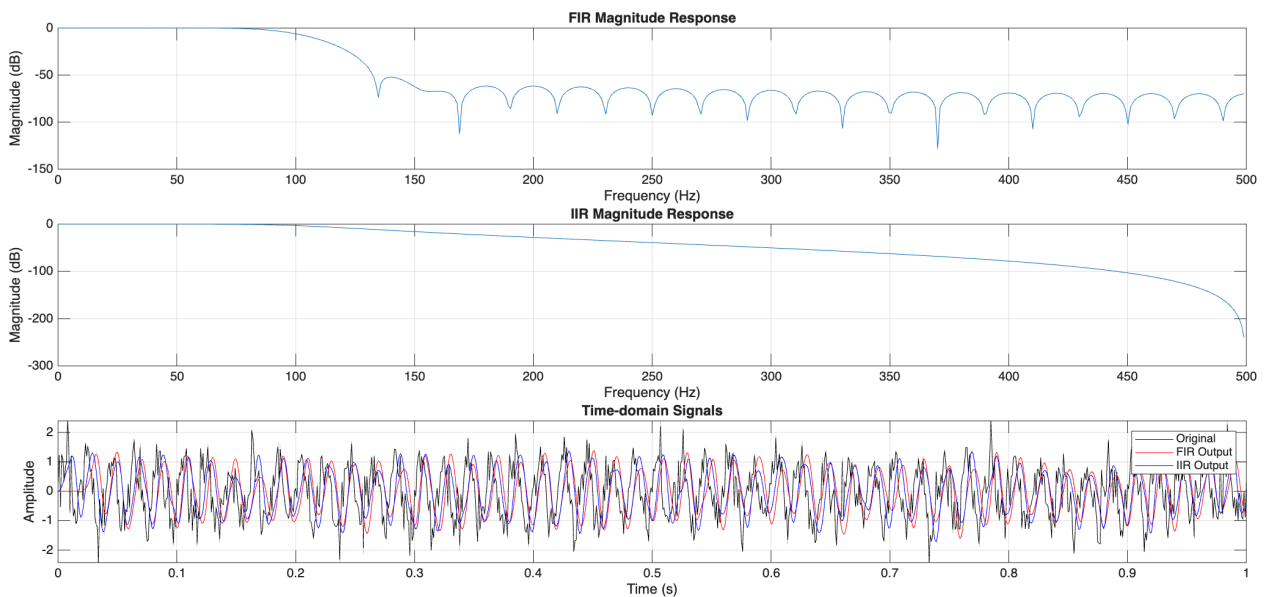


Figure 1: Frequency and Time Domain Responses of FIR and IIR Filters

Analysis

Brief analysis — FIR vs IIR low-pass filters (based on Figure 1)

Summary

- Top subplot (FIR magnitude, dB): clear passband near 0 Hz, a transition band, then periodic deep nulls / stop-band ripples at higher frequencies; passband is fairly flat near 0 dB.
- Middle subplot (IIR / Butterworth magnitude, dB): smooth monotonic roll-off from the passband to deep attenuation without periodic nulls; the curve decreases smoothly and reaches very large negative dB values at high frequencies.
- Bottom subplot (time domain): original (black) overlaid with FIR (red) and IIR (blue). FIR output is visibly delayed but preserves waveform shape; IIR output shows reduced amplitude and some phase/shape differences (nonlinear phase and transient effects).

Key observations

- **FIR magnitude:**

- Linear-phase symmetry yields an approximately constant group delay $\approx N/2$ samples, visible as the time delay in the time-domain plot.
- Windowing sidelobes / spectral leakage produce periodic notches and ripples in the stopband.
- Passband is relatively flat and predictable.

- **IIR magnitude:**

- Provides steeper, monotonic attenuation for a given filter order, yielding stronger stop-band suppression with fewer coefficients.
- Lacks linear phase; phase varies with frequency, causing waveform distortion and altered transient behaviour.
- Extremely large negative dB values at high frequencies are visible (note: numerical flooring was likely applied to avoid $-\infty$ when plotting in dB).

- **Time domain:**

- FIR preserves waveform shape better, with the primary effect being a constant delay.
- IIR introduces amplitude/phase distortion and possible ringing around sharp events; overall output magnitude can be lower for this design.
- Both filters remove high-frequency content, but the FIR output aligns with the input except for delay, while the IIR output shows shape changes due to nonlinear phase.

Discussion

The results confirm the expectations: the windowed FIR (Hamming, order N) exhibits a smooth passband with low sidelobes and a transition width that narrows as N increases, while the Butterworth IIR (order M) attains a steeper roll-off at lower order; the FIR has essentially linear phase with constant group delay $\approx N/2$ preserving waveform shape, whereas the IIR shows nonlinear phase and may introduce transient ringing; numerically, magnitudes were floored before dB conversion to avoid $-\infty$ spikes and the IIR was obtained via the bilinear transform (prewarp when the cutoff is high relative to Nyquist); in practice choose FIR for phase fidelity and IIR for lower complexity/steeper attenuation, or compensate IIR phase / raise FIR order when required.

Conclusion

This experiment demonstrated the complementary strengths of windowed FIR and Butterworth IIR low-pass filters. The FIR (Hamming window) provides linear phase and predictable group delay at the expense of higher order to achieve a narrow transition band. The Butterworth IIR achieves sharper roll-off with lower order and computational cost but at the cost of nonlinear phase and potential transient distortion. Choice of filter should therefore be driven by application requirements: prefer FIR for phase-sensitive signals and IIR for resource-constrained designs needing rapid attenuation. Practical implementations must also account for numerical issues (dB flooring, coefficient quantization) and, for IIRs, pole stability.

References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1999.
- [2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms and Applications*, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2006.
- [3] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2011.